

CMSE 801 - Day 6



Plotting models with matplotlib and using
NumPy to do more complex math

September 16, 2024

Note: We'll try out table assignments starting Wednesday of this week.

General Course Announcements

- Reminder: **Homework 1 is due September 27.**
- Don't forget that we have office hours for you to stop by and get help or talk through the things we're learning! Check the [website](#).
 - If the scheduled times don't work for you, we can schedule by appointment.
- You should have access to the HiTA "AI Assistant" now
 - You are encouraged to experiment with the tool to explore how it can support your learning process in CMSE 801.

Questions/comments from Day 6 PCA

- Can I use colors in plots other than things like "r" or "b"? There are *a lot* of options:
https://matplotlib.org/stable/gallery/color/named_colors.html
- When/where do I have to import modules?
- Will we use anything other than `matplotlib` for making plots?
 - Pandas actually has some [built-in plotting](#) (which is really just matplotlib)
 - There is a package called [seaborn](#) that offers some tools (again, uses matplotlib under the hood)
- One of the more challenging parts seemed to be Question 3 -- **we'll look at that.**
- Some questions related to accessing information from the list in the kindergarten problem -- **we'll check in on that as well.**

Generating a set of finer-grained x values and plotting sine and cosine

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: # Create a list from 0 to 2pi with increments of 0.1
x_list = []
x = 0
dx = 0.1
while x < 2*np.pi:
    x_list.append(x)
    x += dx

# Calculate sine and cosine element-by-elements
sin_xlist = []
cos_xlist = []
for x in x_list:
    sin_xlist.append(np.sin(x))
    cos_xlist.append(np.cos(x))

# Calculating it for the whole list as once (numpy let's us do this!)
sin_xlist = np.sin(x_list)
cos_xlist = np.cos(x_list)
```

In [3]: *# I could have used a NumPy function to create the x values (you'll see*

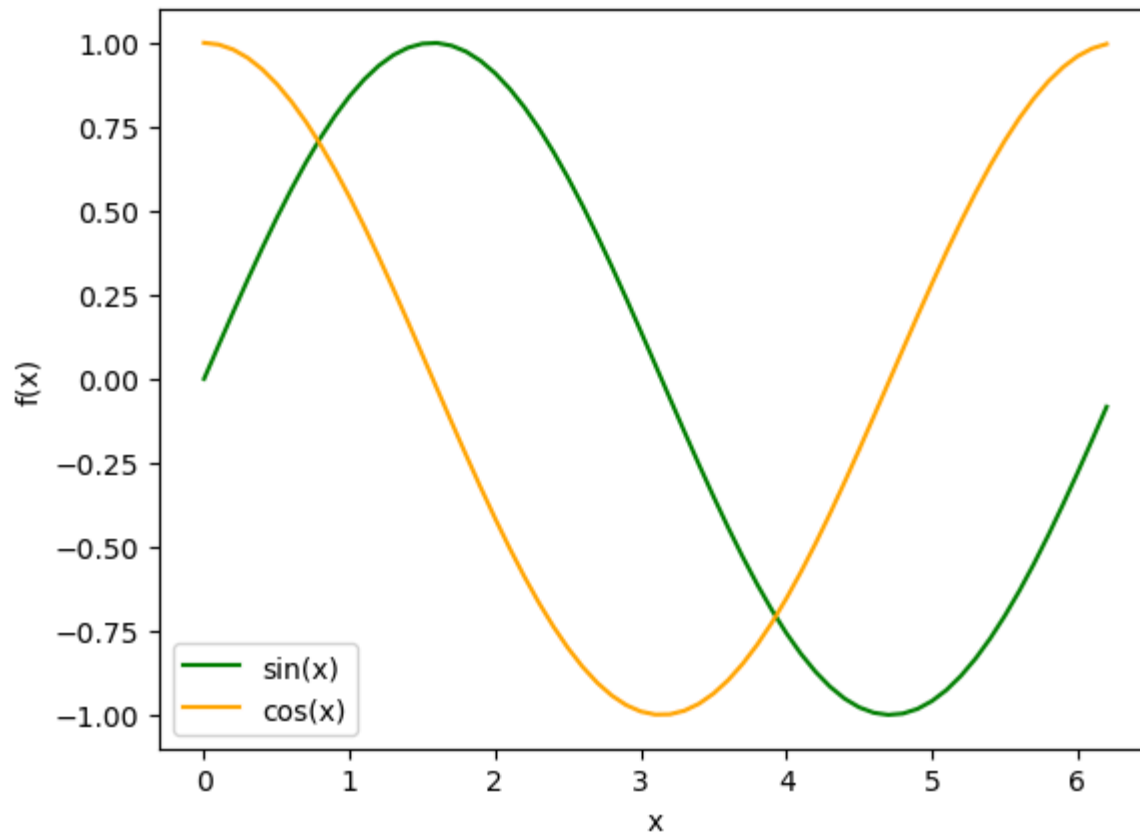
```
dx = 0.1
```

```
x_list = np.arange(0., 2.0 * np.pi, dx)
```

```
print(x_list)
```

```
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.  1.1 1.2 1.3 1.4 1.5
1.6 1.7
1.8 1.9 2.  2.1 2.2 2.3 2.4 2.5 2.6 2.7 2.8 2.9 3.  3.1 3.2 3.3
3.4 3.5
3.6 3.7 3.8 3.9 4.  4.1 4.2 4.3 4.4 4.5 4.6 4.7 4.8 4.9 5.  5.1
5.2 5.3
5.4 5.5 5.6 5.7 5.8 5.9 6.  6.1 6.2]
```

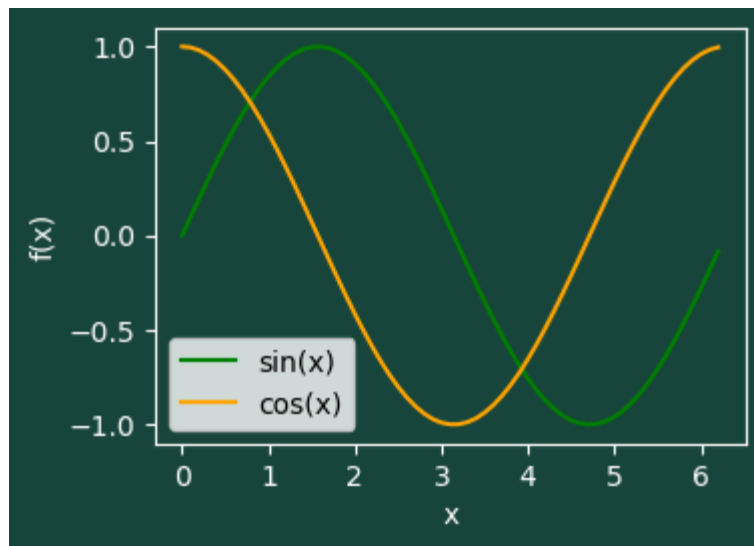
```
In [4]: # Plotting
plt.plot(x_list,sin_xlist,"g",label='sin(x)') # I general try to avoid l
                                              # using "g" instead of us
plt.plot(x_list,cos_xlist,color="orange", label="cos(x)")
plt.legend()
plt.xlabel("x")
plt.ylabel("f(x)");
```



I can also get really carried away with customizing my plots

```
In [5]: # Define some parameters for the plot to customize it, note that this cl
params = {"ytick.color" : "w",
          "xtick.color" : "w",
          "axes.labelcolor" : "w",
          "axes.edgecolor" : "w",
          "axes.facecolor" : "#18453B",
          "legend.facecolor" : "w"}
plt.rcParams.update(params)

# # Plotting
plt.figure(facecolor = "#18453B", figsize = (3.8, 2.7) )
plt.plot(x_list,sin_xlist,"g",label='sin(x)') # Note that a green line :
plt.plot(x_list,cos_xlist,color="orange", label="cos(x)")
plt.legend()
plt.xlabel("x")
plt.ylabel("f(x)");
```



Accessing information from lists inside lists

We'll review the "kindergarten problem"

```
In [9]: children_list = [['Maxine Reed',1], ['Luz Ryan',10], ['Betty Fleming',9],  
    kinder_students = []  
  
    for i in range(len(children_list)):  
        print(children_list[i])
```

['Maxine Reed', 1]
['Luz Ryan', 10]
['Betty Fleming', 9]
['Sherman Powers', 6]
['Jerald Santos', 8]
['Alberta Flowers', 7]
['Lillie Nguyen', 3]
['Martha Jackson', 8]
['Troy Wilkins', 11]
['Eric Mendez', 11]
['Violet Ramsey', 2]
['Beth Thompson', 9]
['Erik Craig', 5]
['Hugo Ray', 12]
['Linda Nash', 2]
['Laura Banks', 12]
['Israel Pierce', 8]
['Dora Caldwell', 5]
['Agnes Matthews', 11]
['Norma Fox', 9]
['Marian Miller', 11]
['Alfredo Cox', 9]
['Manuel Black', 4]
['Beatrice Sanders', 2]
['Bruce Joseph', 5]
['Guadalupe Allison', 2]
['Sheri Carter', 9]
['Floyd Hines', 7]
['Abel Hamilton', 8]
['Angel Jones', 9]
['Carole Webb', 6]

```
['Kelly Brady', 9]
```

```
In [10]: kinder_students = []
         for i in range(len(children_list)):
             # Check month of the student (second item, index = 1)
             if children_list[i][1] > 7:
                 # Append the name of the student (first item, index = 0)
                 kinder_students.append(children_list[i][0])
         print("The following students should be enrolled:")
         for s in kinder_students:
             print(s)
```

The following students should be enrolled:

Luz Ryan
Betty Fleming
Jerald Santos
Martha Jackson
Troy Wilkins
Eric Mendez
Beth Thompson
Hugo Ray
Laura Banks
Israel Pierce
Agnes Matthews
Norma Fox
Marian Miller
Alfredo Cox
Sheri Carter
Abel Hamilton
Angel Jones
Kelly Brady

Goals for today

- Build a computational model for projecting population growth (using functions!)
- Test out a variety of parameters
- Visualize your model using plots in Python!

Looking forward - Day 7 PCA: Working more with NumPy and loading up some data

- More details on how NumPy can be one of the most useful modules/packages in Python
 - Arrays are super handy
- Try loading data for the first time!

